

Deadlock-free Context-free Session Types

Jorge A. Pérez

University of Groningen (The Netherlands)

www.jperez.nl

joint work with

Andreia Mordido

University of Lisbon (Portugal)

(Based on a paper presented at FORTE 2026)

Context

Motivation

- ▶ Software systems rely on the coordination of distributed components.
Goal: Ensuring that interacting processes respect their prescribed **protocols**.

Motivation

- ▶ Software systems rely on the coordination of distributed components.
Goal: Ensuring that interacting processes respect their prescribed **protocols**.
- ▶ In **type-based approaches**, protocols are specified using types;
communication correctness for implementations follows from type checking.

Motivation

- ▶ Software systems rely on the coordination of distributed components.
Goal: Ensuring that interacting processes respect their prescribed **protocols**.
- ▶ In **type-based approaches**, protocols are specified using types;
communication correctness for implementations follows from type checking.
- ▶ Two dimensions:
 - **Protocols**, from finite-state and unstructured to cyclic and richly structured
 - **Correctness**, from basic protocol conformance to (dead)lock freedom

Motivation

- ▶ Software systems rely on the coordination of distributed components.
Goal: Ensuring that interacting processes respect their prescribed **protocols**.
- ▶ In **type-based approaches**, protocols are specified using types;
communication correctness for implementations follows from type checking.
- ▶ Two dimensions:
 - **Protocols**, from finite-state and unstructured to cyclic and richly structured
 - **Correctness**, from basic protocol conformance to (dead)lock freedom

Vision:

Support complex protocols while ensuring the strongest possible guarantees.

Motivation

- ▶ Software systems rely on the coordination of distributed components.
Goal: Ensuring that interacting processes respect their prescribed **protocols**.
- ▶ In **type-based approaches**, protocols are specified using types;
communication correctness for implementations follows from type checking.
- ▶ Two dimensions:
 - **Protocols**, from finite-state and unstructured to cyclic and richly structured
 - **Correctness**, from basic protocol conformance to (dead)lock freedom

Vision:

Support complex protocols while ensuring the strongest possible guarantees.

Our Work: A new type system for higher-order concurrent programs.

- ▶ **Protocols**: Specified using **context-free** session types (tree-structured data)
- ▶ **Correctness**: Protocol conformance + **deadlock freedom** (no stuck programs)

Background

CONCUR'06 Kobayashi:

A New Type System for Deadlock-Free Processes.
(CONCUR Test of Time Award 2026.)

CSL-LICS'14 Padovani:

Deadlock and Lock Freedom in the Linear π -Calculus.

FORTE'15 Padovani and Novara:

Types for Deadlock-Free Higher-Order Programs.

ICFP'16 Thiemann and Vasconcelos:

Context-free Session Types.

LMCS'24 Van den Heuvel and Pérez:

Asynchronous Session-Based Concurrency: Deadlock-freedom in Cyclic Process Networks.

Context-Free Session Types (CFSTs)

- Specify protocols using types with general sequencing:

$$\begin{aligned} S ::= & !S \mid ?S \mid \oplus\{\ell: S_\ell\}_{\ell \in L} \mid \&\{\ell: S_\ell\}_{\ell \in L} \mid \text{Close} \mid \text{Wait} \mid \text{Skip} \\ & \mid \boxed{S_1; S_2} \mid \beta \mid \mu\beta.S \end{aligned}$$

Polymorphism and recursion are essential and entangled.

Context-Free Session Types (CFSTs)

- Specify protocols using types with general sequencing:

$$\begin{aligned} S ::= & !S \mid ?S \mid \oplus\{\ell: S_\ell\}_{\ell \in L} \mid \&\{\ell: S_\ell\}_{\ell \in L} \mid \text{Close} \mid \text{Wait} \mid \text{Skip} \\ & \mid \boxed{S_1; S_2} \mid \beta \mid \mu\beta.S \end{aligned}$$

Polymorphism and recursion are essential and entangled.

- Distinctive example: Serializing a binary tree (with integer nodes):

$$\mu\beta.(\&\{\text{LeafC} : \text{Skip}, \text{NodeC} : ?\text{Int}; \beta; \beta\})$$

This recursive protocol offers (&) two branches.

NodeC first inputs (?) an Int, then inputs the left and right subtrees.

Context-Free Session Types (CFSTs)

- Specify protocols using types with general sequencing:

$$S ::= !S \mid ?S \mid \oplus\{\ell: S_\ell\}_{\ell \in L} \mid \&\{\ell: S_\ell\}_{\ell \in L} \mid \text{Close} \mid \text{Wait} \mid \text{Skip} \\ \mid \boxed{S_1; S_2} \mid \beta \mid \mu\beta.S$$

Polymorphism and recursion are essential and entangled.

- Distinctive example: Serializing a binary tree (with integer nodes):

$$\mu\beta.(\&\{\text{LeafC} : \text{Skip}, \text{NodeC} : ?\text{Int}; \beta; \beta\})$$

This recursive protocol offers (&) two branches.

NodeC first inputs (?) an Int, then inputs the left and right subtrees.

- Prior work builds the meta-theory of CFSTs (subtyping, equivalence, inference) and develops their implementation: the **FreeST programming language**.

Context-Free Session Types (CFSTs)

- Specify protocols using types with general sequencing:

$$S ::= !S \mid ?S \mid \oplus\{\ell: S_\ell\}_{\ell \in L} \mid \&\{\ell: S_\ell\}_{\ell \in L} \mid \text{Close} \mid \text{Wait} \mid \text{Skip} \\ \mid \boxed{S_1; S_2} \mid \beta \mid \mu\beta.S$$

Polymorphism and recursion are essential and entangled.

- Distinctive example: Serializing a binary tree (with integer nodes):

$$\mu\beta.(\&\{\text{LeafC} : \text{Skip}, \text{NodeC} : ?\text{Int}; \beta; \beta\})$$

This recursive protocol offers (&) two branches.

NodeC first inputs (?) an Int, then inputs the left and right subtrees.

- Prior work builds the meta-theory of CFSTs (subtyping, equivalence, inference) and develops their implementation: the **FreeST programming language**.
- **Limitation:** Existing CFSTs accept **deadlocked processes as typable**.

Contributions

1. A call-by-value λ -calculus with concurrency, equipped with sessions.
Expressions are organized in **configurations** where communication occurs.
2. A type system for (1) based on CFSTs with polymorphic recursion, and a proof that **well-typed configurations are deadlock-free**.

Contributions

1. A call-by-value λ -calculus with concurrency, equipped with sessions. Expressions are organized in **configurations** where communication occurs.
2. A type system for (1) based on CFSTs with polymorphic recursion, and a proof that **well-typed configurations are deadlock-free**.

Key features:

- ▶ CFSTs + **priorities** into polymorphic functions and recursive sessions. This ensures deadlock freedom for programs with **cyclic process networks**.
- ▶ Our design is well-aligned with the FreeST programming language.

Contributions

1. A call-by-value λ -calculus with concurrency, equipped with sessions. Expressions are organized in **configurations** where communication occurs.
2. A type system for (1) based on CFSTs with polymorphic recursion, and a proof that **well-typed configurations are deadlock-free**.

Key features:

- ▶ CFSTs + **priorities** into polymorphic functions and recursive sessions. This ensures deadlock freedom for programs with **cyclic process networks**.
- ▶ Our design is well-aligned with the FreeST programming language.

This talk: Our framework by example, with a brief view on the theory.

Deadlocks

A Classic Deadlock Scenario

A program with two processes, which share two channels of type $T = ?\text{Int}; \text{Wait}$.

A single process:

```
1  f : T → dualof T → ()  
2  f r w = let (n, r) = receive r in  
3          let w = send n w in  
4          wait r; close w
```

The whole program:

```
1  main : ()  
2  main =  
3    let (r1, w1) = new @T () in  
4    let (r2, w2) = new @T () in  
5    fork (\_ : () → f r1 w2);  
6    f r2 w1
```

A Classic Deadlock Scenario

A program with two processes, which share two channels of type $T = ?\text{Int}; \text{Wait}$.

A single process:

Function signature: receive two endpoints, return unit

```
1 f : T → dualof T → ()  
2 f r w = let (n, r) = receive r in  
3         let w = send n w in  
4         wait r; close w
```

The whole program:

```
1 main : ()  
2 main =  
3   let (r1, w1) = new @T () in  
4   let (r2, w2) = new @T () in  
5   fork (\_ : () → f r1 w2);  
6   f r2 w1
```

A Classic Deadlock Scenario

A program with two processes, which share two channels of type $T = ?\text{Int}; \text{Wait}$.

A single process:

```
1  f : T → dualof T → ()
2  f r w = let (n, r) = receive r in
3          let w = send n w in
4          wait r; close w
```

The dual of type T : **!Int; Close**

The whole program:

```
1  main : ()
2  main =
3    let (r1, w1) = new @T () in
4    let (r2, w2) = new @T () in
5    fork (\_ : () → f r1 w2);
6    f r2 w1
```

A Classic Deadlock Scenario

A program with two processes, which share two channels of type $T = ?\text{Int}; \text{Wait}$.

A single process:

```
1 f : T → dualof T → ()  
2 f r w = let (n, r) = receive r in  
3         let w = send n w in  
4         wait r; close w
```

A linear function



The whole program:

```
1 main : ()  
2 main =  
3   let (r1, w1) = new @T () in  
4   let (r2, w2) = new @T () in  
5   fork (\_ : () → f r1 w2);  
6   f r2 w1
```

A Classic Deadlock Scenario

A program with two processes, which share two channels of type $T = ?\text{Int}; \text{Wait}$.

A single process:

```
1 f : T → dualof T → ()  
2 f r w = let (n, r) = receive r in  
3       let w = send n w in  
4       wait r; close w
```

First input on endpoint r,
then output on w

The whole program:

```
1 main : ()  
2 main =  
3   let (r1, w1) = new @T () in  
4   let (r2, w2) = new @T () in  
5   fork (\_ : () → f r1 w2);  
6   f r2 w1
```

A Classic Deadlock Scenario

A program with two processes, which share two channels of type $T = ?\text{Int}; \text{Wait}$.

A single process:

```
1  f : T → dualof T → ()  
2  f r w = let (n, r) = receive r in  
3          let w = send n w in  
4          wait r; close w
```

The whole program:

```
1  main : ()  
2  main =  
3    let (r1, w1) = new @T () in  
4    let (r2, w2) = new @T () in  
5    fork (\_ : () → f r1 w2);  
6    f r2 w1
```

A Classic Deadlock Scenario

A program with two processes, which share two channels of type $T = ?\text{Int}; \text{Wait}$.

A single process:

```
1 f : T → dualof T → ()
2 f r w = let (n, r) = receive r in
3         let w = send n w in
4         wait r; close w
```

The whole program:

```
1 main : ()
2 main =
3   let (r1, w1) = new @T () in
4   let (r2, w2) = new @T () in
5   fork (\_ : () → f r1 w2);
6   f r2 w1
```



Creates one pair of endpoints
of type T (and its dual)

A Classic Deadlock Scenario

A program with two processes, which share two channels of type $T = ?\text{Int}; \text{Wait}$.

A single process:

```
1 f : T → dualof T → ()  
2 f r w = let (n, r) = receive r in  
3         let w = send n w in  
4         wait r; close w
```

The whole program:

```
1 main : ()  
2 main =  
3   let (r1, w1) = new @T () in  
4   let (r2, w2) = new @T () in  
5   fork (\_ : () → f r1 w2);  
6   f r2 w1
```

Spawns two parallel
instances of process f

A Classic Deadlock Scenario

A program with two processes, which share two channels of type $T = ?\text{Int}; \text{Wait}$.

A single process:

```
1 f : T → dualof T → ()  
2 f r w = let (n, r) = receive r in  
3         let w = send n w in  
4         wait r; close w
```

The whole program:

```
1 main : ()  
2 main =  
3   let (r1, w1) = new @T () in  
4   let (r2, w2) = new @T () in  
5   fork (\_ : () → f r1 w2);  
6   f r2 w1
```



Deadlock!

Priorities to the Rescue

Priorities for Deadlock Freedom

- ▶ Key idea: Each action has a **priority**: an integer that denotes urgency/order. (Smaller number = highest priority.)
- ▶ Immediately enabled actions have a higher priority than those appearing later.
- ▶ Circular dependencies between threads $\Rightarrow$ No valid priority assignment.

Priorities for Deadlock Freedom

- ▶ Key idea: Each action has a **priority**: an integer that denotes urgency/order. (Smaller number = highest priority.)
- ▶ Immediately enabled actions have a higher priority than those appearing later.
- ▶ Circular dependencies between threads $\Rightarrow$ No valid priority assignment.

Example

Take $p, q \geq 0$. We can revise $T = ?\text{Int}; \text{Wait}$ into

```
type R = ? $p$  Int ; Wait ( $p+2$ )
```

```
type S = ? $q$  Int ; Wait ( $q+2$ )
```

Duality preserves priorities:

```
dualof R = ! $p$  Int ; Close ( $p+2$ )
```

Priorities for Deadlock Freedom

- ▶ In a functional setting, i/o behavior arises via constants (with functional types), but not via explicit communication actions. How to specify priorities?

Priorities for Deadlock Freedom

- ▶ In a functional setting, i/o behavior arises via constants (with functional types), but not via explicit communication actions. How to specify priorities?
- ▶ Padovani & Novara (2015): Arrow types with **boundaries** for i/o behavior:

$$T \rightarrow_m^{\rho_1, \rho_2} U$$

where ρ_1, ρ_2 denote integers extended with $\perp$ and $\top$. We have:

Priorities for Deadlock Freedom

- ▶ In a functional setting, i/o behavior arises via constants (with functional types), but not via explicit communication actions. How to specify priorities?
- ▶ Padovani & Novara (2015): Arrow types with **boundaries** for i/o behavior:

$$T \rightarrow_m^{\rho_1, \rho_2} U$$

where ρ_1, ρ_2 denote integers extended with $\perp$ and $\top$. We have:

- ▶ ρ_1 is a **lower bound**

The function contains channels with priority ρ_1 or greater

Priorities for Deadlock Freedom

- ▶ In a functional setting, i/o behavior arises via constants (with functional types), but not via explicit communication actions. How to specify priorities?
- ▶ Padovani & Novara (2015): Arrow types with **boundaries** for i/o behavior:

$$T \rightarrow_m^{\rho_1, \rho_2} U$$

where ρ_1, ρ_2 denote integers extended with $\perp$ and $\top$. We have:

- ▶ ρ_1 is a **lower bound**
The function contains channels with priority ρ_1 or greater
- ▶ ρ_2 is an **upper bound**
The maximum priority triggered once the function is applied (a latent effect)

Priorities for Deadlock Freedom

- ▶ In a functional setting, i/o behavior arises via constants (with functional types), but not via explicit communication actions. How to specify priorities?
- ▶ Padovani & Novara (2015): Arrow types with **boundaries** for i/o behavior:

$$T \rightarrow_m^{\rho_1, \rho_2} U$$

where ρ_1, ρ_2 denote integers extended with $\perp$ and $\top$. We have:

- ▶ ρ_1 is a **lower bound**
The function contains channels with priority ρ_1 or greater
- ▶ ρ_2 is an **upper bound**
The maximum priority triggered once the function is applied (a latent effect)
- ▶ m is the function's **multiplicity**: linear (to be used exactly once) or unrestricted

Priorities for Deadlock Freedom

- ▶ In a functional setting, i/o behavior arises via constants (with functional types), but not via explicit communication actions. How to specify priorities?
- ▶ Padovani & Novara (2015): Arrow types with **boundaries** for i/o behavior:

$$T \rightarrow_m^{\rho_1, \rho_2} U$$

where ρ_1, ρ_2 denote integers extended with $\perp$ and $\top$. We have:

- ▶ ρ_1 is a **lower bound**
 - ▶ ρ_2 is an **upper bound**
 - ▶ m is the function's **multiplicity**
- ▶ This way, e.g., $T \rightarrow_m^{\top, \perp} U$ is the type of pure functions (no i/o behavior).

Priorities for Deadlock Freedom

```
1  type R = ?pInt; Wait (p+2)
2  type S = ?qInt; Wait (q+2)
3
4  f : R → [T, ⊥] dualof S ⇝ [p, q+2] ()
5  f r w =
6    let (n, r) = receive r in -- priority: p
7    let w = send n w in      -- priority: q
8    wait r;                  -- priority: p+2
9    close w                  -- priority: q+2
10 g : S → [T, ⊥] dualof R ⇝ [q, p+2] ()
11 g r w =
12   let (n, r) = receive r in -- priority: q
13   let w = send n w in      -- priority: p
14   ...
```

Priorities for Deadlock Freedom

T : no channel in the environment
 $\perp$: no latent effect after application

```
1  type R = ?pInt; Wait (p+2)
2  type S = ?qInt; Wait (q+2)
3
4  f : R  $\rightarrow$  [T,  $\perp$ ] dualof S  $\multimap$  [p, q+2] ()
5  f r w =
6    let (n, r) = receive r in -- priority: p
7    let w = send n w in      -- priority: q
8    wait r;                  -- priority: p+2
9    close w                  -- priority: q+2
10 g : S  $\rightarrow$  [T,  $\perp$ ] dualof R  $\multimap$  [q, p+2] ()
11 g r w =
12   let (n, r) = receive r in -- priority: q
13   let w = send n w in      -- priority: p
14   ...
```

Priorities for Deadlock Freedom

```
1  type R = ?pInt ; Wait (p+2)
2  type S = ?qInt ; Wait (q+2)
3
4  f : R → [T, ⊥] dualof S → [p, q+2] ()
5  f r w =
6    let (n, r) = receive r in -- priority: p
7    let w = send n w in      -- priority: q
8    wait r;                  -- priority: p+2
9    close w                  -- priority: q+2
10 g : S → [T, ⊥] dualof R → [q, p+2] ()
11 g r w =
12   let (n, r) = receive r in -- priority: q
13   let w = send n w in      -- priority: p
14   ...
```

p: lin channel x in the environment
q+2: max priority after application



Priorities for Deadlock Freedom

```
1  type R = ?pInt; Wait (p+2)
2  type S = ?qInt; Wait (q+2)
3
4  f : R → [T, ⊥] dualof S → [p, q+2] ()
5  f r w =
6    let (n, r) = receive r in -- priority: p
7    let w = send n w in      -- priority: q
8    wait r;                  -- priority: p+2
9    close w                  -- priority: q+2
10 g : S → [T, ⊥] dualof R → [q, p+2] ()
11 g r w =
12   let (n, r) = receive r in -- priority: q
13   let w = send n w in      -- priority: p
14   ...
```

Priorities for Deadlock Freedom

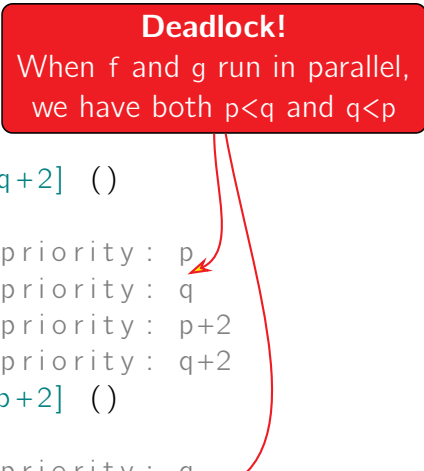
```
1  type R = ?pInt; Wait (p+2)
2  type S = ?qInt; Wait (q+2)
3
4  f : R → [T, ⊥] dualof S → [p, q+2] ()
5  f r w =
6    let (n, r) = receive r in -- priority: p
7    let w = send n w in      -- priority: q
8    wait r;                  -- priority: p+2
9    close w                  -- priority: q+2
10 g : S → [T, ⊥] dualof R → [q, p+2] ()
11 g r w =
12   let (n, r) = receive r in -- priority: q
13   let w = send n w in      -- priority: p
14   ...
```

Priorities for Deadlock Freedom

```
1  type R = ?pInt; Wait (p+2)
2  type S = ?qInt; Wait (q+2)
3
4  f : R → [T, ⊥] dualof S ⇝ [p, q+2] ()
5  f r w =
6    let (n, r) = receive r in -- priority: p
7    let w = send n w in      -- priority: q
8    wait r;                  -- priority: p+2
9    close w                   -- priority: q+2
10 g : S → [T, ⊥] dualof R ⇝ [q, p+2] ()
11 g r w =
12   let (n, r) = receive r in -- priority: q
13   let w = send n w in      -- priority: p
14   ...
```

Deadlock!

When f and g run in parallel,
we have both $p < q$ and $q < p$



The Issue and Our Contributions

The Interplay of Priorities and Recursion (1/2)

Consider the following function, with two **streams**:

```
1 client c1 c2 =  
2   let c1 = send () c1 in  
3   let (_, c2) = receive c2 in  
4   client c1 c2
```

The Interplay of Priorities and Recursion (1/2)

Consider the following function, with two **streams**:

```
1 client c1 c2 =  
2   let c1 = send () c1 in  
3   let (_, c2) = receive c2 in  
4   client c1 c2
```

Take the (equi)recursive types

```
type Stream1 = !p() ; Stream1  
type Stream2 = !q() ; Stream2
```

We can then type the function as follows

$$\text{client} : \text{Stream1} \rightarrow [\top, \perp] \text{ dualof } \text{Stream2} \multimap [\text{p}, \top] ()$$

The Interplay of Priorities and Recursion (1/2)

Consider the following function, with two **streams**:

```
1 client c1 c2 =  
2   let c1 = send () c1 in  
3   let (_, c2) = receive c2 in  
4   client c1 c2
```

Take the (equi)recursive types

```
type Stream1 = !p() ; Stream1  
type Stream2 = !q() ; Stream2
```

We can then type the function as follows

```
client : Stream1  $\rightarrow$   $[T, \perp]$  dualof Stream2  $\multimap$   $[p, T]$  ()
```

Problem: Reusing priorities leads to inconsistencies!

The Interplay of Priorities and Recursion (2/2)

To see the issue, consider the following program:

```
1 altClient : Stream1 → [T,⊥] dualof Stream2 → [p,T] ()
2 altClient c1 c2 =
3   -- first send/receive
4   let c1 = send () c1 in      -- priority: p
5   let (_,c2) = receive c2 in  -- priority: q
6   -- second send/receive
7   let c1 = send () c1 in      -- priority: p (!)
8   let (_,c2) = receive c2 in  -- priority: q (!)
9   altClient c1 c2
```

The Interplay of Priorities and Recursion (2/2)

To see the issue, consider the following program:

```
1  altClient : Stream1 → [T,⊥] dualof Stream2 → [p,T] ()
2  altClient c1 c2 =
3    -- first send/receive
4    let c1 = send () c1 in           -- priority: p
5    let (_,c2) = receive c2 in      -- priority: q
6    -- second send/receive
7    let c1 = send () c1 in           -- priority: p (!)
8    let (_,c2) = receive c2 in      -- priority: q (!)
9    altClient c1 c2
```

In Lines 8-9, equi-recursion re-introduces priorities p and q.

The program is **ill-typed**!

Our Novel Contributions

We extend functional types $(T, U, \dots)$ and session types $(S_1, S_2, \dots)$ with

1. **Polymorphic recursion** on types **and** priorities

$\forall \alpha^{\rho}. T$	(polymorphic functional type)
$\forall \beta^{\rho}. S$	(polymorphic session type)

Our Novel Contributions

We extend functional types $(T, U, \dots)$ and session types $(S_1, S_2, \dots)$ with

1. **Polymorphic recursion** on types **and** priorities

$$\begin{array}{ll} \forall \alpha^{\rho}. T & \text{(polymorphic functional type)} \\ \forall \beta^{\rho}. S & \text{(polymorphic session type)} \end{array}$$

2. **Priority abstractions** bind a **priority variable** ι to a **priority interval** φ

$$\begin{array}{ll} \forall^F_{\iota^{\varphi}}. T & \text{(functional)} \\ \forall^S_{\iota^{\varphi}}. S & \text{(session)} \end{array}$$

Our Novel Contributions

We extend functional types $(T, U, \dots)$ and session types $(S_1, S_2, \dots)$ with

1. **Polymorphic recursion** on types **and** priorities

$$\begin{array}{ll} \forall \alpha^{\rho}. T & \text{(polymorphic functional type)} \\ \forall \beta^{\rho}. S & \text{(polymorphic session type)} \end{array}$$

2. **Priority abstractions** bind a **priority variable** ι to a **priority interval** φ

$$\begin{array}{ll} \forall^F_{\iota^{\varphi}}. T & \text{(functional)} \\ \forall^S_{\iota^{\varphi}}. S & \text{(session)} \end{array}$$

3. **Priority sequences**: a ‘stack’ of integers, used for priority instantiation.
There is a sequence per channel, defined upon endpoint creation (**new**).

The Stream Example, Revisited (1/4)

```
1 type Stream =  $\forall i \in (\perp, \top) \Rightarrow !i()$  ; Stream
2 client:  $\forall p \in (\perp, \top) \Rightarrow \text{Stream} \rightarrow [\top, \perp]$  dual of Stream
3  $\multimap [p, \top] ()$ 
```

The Stream Example, Revisited (1/4)

Priority-quantified **session**

```
1 type Stream =  $\forall i \in (\perp, \top) \Rightarrow !i()$  ; Stream  
2 client :  $\forall p \in (\perp, \top) \Rightarrow \text{Stream} \rightarrow [\top, \perp]$  dual of Stream  
3  $\multimap [p, \top] ()$ 
```



The Stream Example, Revisited (1/4)

Priority-quantified **session**

```
1 type Stream =  $\forall i \in (\perp, \top) \Rightarrow !i()$  ; Stream  
2 client:  $\forall p \in (\perp, \top) \Rightarrow \text{Stream} \rightarrow [\top, \perp]$  dual of Stream  
3  $\multimap [p, \top] ()$ 
```

Priority-quantified **function**

The Stream Example, Revisited (2/4)

```
1 type Stream =  $\forall i \in (\perp, T) \Rightarrow !i()$  ; Stream
2 client:  $\forall p \in (\perp, T) \Rightarrow \text{Stream} \rightarrow [T, \perp]$  dual of Stream
3                                      $\multimap [p, T] ()$ 
4 client c1 c2 =
5     ...
```

How to express the concrete priorities for polymorphic endpoints and functions?

The Stream Example, Revisited (2/4)

```
1 type Stream =  $\forall i \in (\perp, T) \Rightarrow !i()$  ; Stream
2 client:  $\forall p \in (\perp, T) \Rightarrow \text{Stream} \rightarrow [T, \perp]$  dual of Stream
3                                            $\multimap [p, T] ()$ 
4 client c1 c2 =
5     ...
```

How to express the concrete priorities for polymorphic endpoints and functions?

Our Solution: Three constructs in the program syntax:

The Stream Example, Revisited (2/4)

```
1 type Stream =  $\forall i \in (\perp, T) \Rightarrow !i()$  ; Stream
2 client:  $\forall p \in (\perp, T) \Rightarrow$  Stream  $\rightarrow [T, \perp]$  dualof Stream
3                                            $\multimap [p, T]$  ()
4 client c1 c2 =
5     ...
```

How to express the concrete priorities for polymorphic endpoints and functions?

Our Solution: Three constructs in the program syntax:

- ▶ '**inst** c' $\rightsquigarrow$ Use endpoint c with the first priority in its sequence, popping it

The Stream Example, Revisited (2/4)

```
1 type Stream =  $\forall i \in (\perp, T) \Rightarrow !i()$  ; Stream
2 client:  $\forall p \in (\perp, T) \Rightarrow \text{Stream} \rightarrow [T, \perp]$  dualof Stream
3                                            $\multimap [p, T]$  ()
4 client c1 c2 =
5     ...
```

How to express the concrete priorities for polymorphic endpoints and functions?

Our Solution: Three constructs in the program syntax:

- ▶ **'inst c'** $\rightsquigarrow$ Use endpoint c with the first priority in its sequence, popping it
- ▶ **'foo{p}'** $\rightsquigarrow$ Instantiate function foo with priority p

The Stream Example, Revisited (2/4)

```
1 type Stream =  $\forall i \in (\perp, T) \Rightarrow !i()$  ; Stream
2 client:  $\forall p \in (\perp, T) \Rightarrow \text{Stream} \rightarrow [T, \perp]$  dualof Stream
3                                      $\multimap [p, T]$  ()
4 client c1 c2 =
5     ...
```

How to express the concrete priorities for polymorphic endpoints and functions?

Our Solution: Three constructs in the program syntax:

- ▶ '**inst** c' $\rightsquigarrow$ Use endpoint c with the first priority in its sequence, popping it
- ▶ 'foo{p}' $\rightsquigarrow$ Instantiate function foo with priority p
- ▶ '**next** c' $\rightsquigarrow$ Read the next priority in the sequence of c, but without popping it

The Stream Example, Revisited (3/4)

```
1 type Stream =  $\forall i \in (\perp, \top) \Rightarrow !i()$  ; Stream
2 client:  $\forall p \in (\perp, \top) \Rightarrow \text{Stream} \rightarrow [\top, \perp]$  dual of Stream
3                                      $\multimap [p, \top] ()$ 
4 client c1 c2 =
5   let c1 = send () (inst c1) in
6   let (_, c2) = receive (inst c2) in
7   client{next c1} c1 c2
```

The Stream Example, Revisited (3/4)

Priority-quantified **function**

Priority-quantified **session**

```
1 type Stream =  $\forall i \in (\perp, T) \Rightarrow !i() ; \text{Stream}$ 
2 client:  $\forall p \in (\perp, T) \Rightarrow \text{Stream} \rightarrow [T, \perp]$  dual of Stream
3                                      $\multimap [p, T] ()$ 
4 client c1 c2 =
5   let c1 = send () (inst c1) in
6   let (_, c2) = receive (inst c2) in
7   client{next c1} c1 c2
```

The Stream Example, Revisited (3/4)

Priority-quantified **function**

Priority-quantified **session**

```
1 type Stream =  $\forall i \in (\perp, T) \Rightarrow !i() ; \text{Stream}$   
2 client:  $\forall p \in (\perp, T) \Rightarrow \text{Stream} \rightarrow [T, \perp]$  dual of Stream  
3  $\multimap [p, T] ()$   
4 client c1 c2 =  
5   let c1 = send () (inst c1) in  
6   let (_, c2) = receive (inst c2) in  
7   client{next c1} c1 c2
```

Pop priority for the
polymorphic **session**

The Stream Example, Revisited (3/4)

Priority-quantified **function**

Priority-quantified **session**

```
1 type Stream =  $\forall i \in (\perp, T) \Rightarrow !i() ; \text{Stream}$ 
2 client:  $\forall p \in (\perp, T) \Rightarrow \text{Stream} \rightarrow [T, \perp]$  dual of Stream
3  $\multimap [p, T] ()$ 
4 client c1 c2 =
5   let c1 = send () (inst c1) in
6   let (_, c2) = receive (inst c2) in
7   client {next c1} c1 c2
```

Pop priority for the
polymorphic **session**

Use {...} to instantiate the
polymorphic **function**

The Stream Example, Revisited (4/4)

To wrap up our example, we must specify the **priority sequences**.

Each sequence is defined via **new** in terms of an **start value** and an **increment**:

```
1  main : ()  
2  main =  
3      let (w1, r1) = new Stream 1 2 () in  
4      let (w2, r2) = new Stream 2 2 () in  
5      fork (\_ : () → client {next w1} w1 r2);  
6      newclient {next w2} w2 r1
```

The Stream Example, Revisited (4/4)

To wrap up our example, we must specify the **priority sequences**.

Each sequence is defined via **new** in terms of an **start value** and an **increment**:

```
1  main : ()
2  main =
3    let (w1, r1) = new Stream 1 2 () in
4    let (w2, r2) = new Stream 2 2 () in
5    fork (\_ : () → client { next w1 } w1 r2);
6    newclient { next w2 } w2 r1
```

Start: 1, Increment: 2
Sequence: [1, 3, 5, ...]

The Stream Example, Revisited (4/4)

To wrap up our example, we must specify the **priority sequences**.

Each sequence is defined via **new** in terms of an **start value** and an **increment**:

```
1  main : ()
2  main =
3    let (w1, r1) = new Stream 1 2 () in
4    let (w2, r2) = new Stream 2 2 () in
5    fork (\_ : () → client { next w1 } w1 r2);
6    newclient { next w2 } w2 r1
```

Start: 1, Increment: 2
Sequence: [1, 3, 5, ...]

Start: 2, Increment: 2
Sequence: [2, 4, 6, ...]

The Stream Example, Revisited (4/4)

To wrap up our example, we must specify the **priority sequences**.

Each sequence is defined via **new** in terms of an **start value** and an **increment**:

```
1  main : ()
2  main =
3    let (w1, r1) = new Stream 1 2 () in
4    let (w2, r2) = new Stream 2 2 () in
5    fork (\_:() → client { next w1 } w1 r2);
6    newclient { next w2 } w2 r1
```

Start: 1, Increment: 2
Sequence: [1, 3, 5, ...]

Start: 2, Increment: 2
Sequence: [2, 4, 6, ...]

Similar to client,
but with send/receive swapped

Deadlock-Free CFSTs in Action

Deadlock-Free CFSTs in Action

- Recall the type for serializing a binary tree:

$$\mu\beta.(\&\{\text{LeafC} : \text{Skip}, \text{NodeC} : ?\text{Int}; \beta; \beta\})$$

Deadlock-Free CFSTs in Action

- ▶ Recall the type for serializing a binary tree:

$$\mu\beta.(\&\{\text{LeafC} : \text{Skip}, \text{NodeC} : ?\text{Int}; \beta; \beta\})$$

- ▶ In our type system, we define:

$$\text{TreeChan} = \mu\beta.(\forall^S_{\iota}(\perp, \top). \&^{\iota}\{\text{LeafC} : \text{Skip}, \text{NodeC} : ?^{\iota+1}\text{Int}; \beta; \beta\}).$$

A channel first instantiates the priority variable ι and then offers the branches.
The integer is received under priority $\iota + 1$.

Deadlock-Free CFSTs in Action

- ▶ Recall the type for serializing a binary tree:

$$\mu\beta.(\&\{\text{LeafC} : \text{Skip}, \text{NodeC} : ?\text{Int}; \beta; \beta\})$$

- ▶ In our type system, we define:

$$\text{TreeChan} = \mu\beta.(\forall^S_{\iota}(\perp, \top). \&\iota\{\text{LeafC} : \text{Skip}, \text{NodeC} : ?^{\iota+1}\text{Int}; \beta; \beta\}).$$

A channel first instantiates the priority variable ι and then offers the branches.
The integer is received under priority $\iota + 1$.

- ▶ Define trees as follows:

```
data Tree = Leaf
          | Node Int Tree Tree
```

We use TreeChan to implement a function recvTree:

$$\text{recvTree} : \forall a \Rightarrow \text{TreeChan}; a \rightarrow (\text{Tree}, a)$$

Deadlock-Free CFSTs in Action

```
1  recvTree :  $\forall p \in (\perp, T) \Rightarrow \forall a :: p \Rightarrow$   
2              TreeChan; a  $\rightarrow [T, T]$  (Tree, a)  
3  recvTree c =  
4      match (inst c) with {  
5          LeafC c  $\rightarrow$   
6              (Leaf, c),  
7          NodeC c  $\rightarrow$   
8              let (x, c) = receive c in  
9              let (left, c) = recvTree{next c} @(TreeChan;a) c in  
10             let (right, c) = recvTree{next c} @a c in  
11             (Node x left right, c) }
```

Deadlock-Free CFSTs in Action

A **variable** for a type with priority p

```
1  recvTree :  $\forall p \in (\perp, T) \Rightarrow \forall a :: p \Rightarrow$   
2              TreeChan;  $a \rightarrow [T, T] \text{ (Tree, a)}$   
3  recvTree c =  
4      match (inst c) with {  
5          LeafC c  $\rightarrow$   
6              (Leaf, c),  
7          NodeC c  $\rightarrow$   
8              let (x, c) = receive c in  
9              let (left, c) = recvTree{next c} @(TreeChan; a) c in  
10             let (right, c) = recvTree{next c} @a c in  
11             (Node x left right, c) }
```

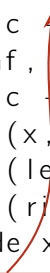
Returns a **pair**: tree + continuation

Deadlock-Free CFSTs in Action

```
1  recvTree :  $\forall p \in (\perp, T) \Rightarrow \forall a :: p \Rightarrow$   
2              TreeChan; a  $\rightarrow [T, T]$  (Tree, a)  
3  recvTree c =  
4      match (inst c) with {  
5      LeafC c  $\rightarrow$   
6          (Leaf, c),  
7      NodeC c  $\rightarrow$   
8          let (x, c) = receive c in  
9          let (left, c) = recvTree{next c} @(TreeChan;a) c in  
10         let (right, c) = recvTree{next c} @a c in  
11         (Node x left right, c) }
```


Deadlock-Free CFSTs in Action

```
1  recvTree :  $\forall p \in (\perp, T) \Rightarrow \forall a :: p \Rightarrow$   
2              TreeChan; a  $\rightarrow [T, T]$  (Tree, a)  
3  recvTree c =  
4      match (inst c) with {  
5          LeafC c  $\rightarrow$   
6              (Leaf, c),  
7          NodeC c  $\rightarrow$   
8              let (x, c) = receive c in  
9              let (left, c) = recvTree{next c} @(TreeChan;a) c in  
10             let (right, c) = recvTree{next c} @a c in  
11             (Node x left right, c) }
```



Pop the priority sequence

Deadlock-Free CFSTs in Action

```
1  recvTree :  $\forall p \in (\perp, T) \Rightarrow \forall a :: p \Rightarrow$   
2              TreeChan; a  $\rightarrow [T, T]$  (Tree, a)  
3  recvTree c =  
4      match (inst c) with {  
5          LeafC c  $\rightarrow$   
6              (Leaf, c),  
7          NodeC c  $\rightarrow$   
8              let (x, c) = receive c in  
9              let (left, c) = recvTree{next c} @(TreeChan;a) c in  
10             let (right, c) = recvTree{next c} @a c in  
11             (Node x left right, c) }
```

Pop the priority sequence

Instantiate with a priority ($\{...\}$) **and** a type ($@$)

Deadlock-Free CFSTs in Action

```
1  recvTree :  $\forall p \in (\perp, T) \Rightarrow \forall a :: p \Rightarrow$   
2              TreeChan; a  $\rightarrow [T, T]$  (Tree, a)  
3  recvTree c =  
4      match (inst c) with {  
5          LeafC c  $\rightarrow$   
6              (Leaf, c),  
7          NodeC c  $\rightarrow$   
8              let (x, c) = receive c in  
9              let (left, c) = recvTree{next c}@ (TreeChan; a) c in  
10             let (right, c) = recvTree{next c} @a c in  
11             (Node x left right, c) }
```

Two occurrences of 'next c'
but with different values!



Some Technical Details

Syntax: Constants

$\sigma ::= \rho \mid \text{next}^k x$ (Priority values)

$c ::= \text{send} \mid \text{receive} \mid \text{select } k \mid \text{close} \mid \text{wait} \mid \text{fork} \mid \text{fix} \mid ()$ (Constants)

$v ::= x \mid c \mid \lambda_m x : T.e \mid (v, v)$ (Values)

$\mid \Lambda \alpha.v \mid \Lambda \ell.x \mid \text{close}\{\sigma\} \mid \text{wait}\{\sigma\}$

$\mid \text{fork}\{\sigma_1\} \mid \text{fork}\{\sigma_1\}\{\sigma_2\}$

$\mid \text{send}\{\sigma_1\} \mid \text{send}\{\sigma_1\}[T] \mid \dots$

$\mid \text{receive}\{\sigma_1\} \mid \text{receive}\{\sigma_1\}[T] \mid \dots$

$\mid x\{\sigma\}$

Syntax: Constants and Values

$\sigma ::= \rho \mid \text{next}^k x$ (Priority values)

$c ::= \text{send} \mid \text{receive} \mid \text{select } k \mid \text{close} \mid \text{wait} \mid \text{fork} \mid \text{fix} \mid ()$ (Constants)

$v ::= x \mid c \mid \lambda_m x : T.e \mid (v, v)$ (Values)

$\mid \Lambda \alpha.v \mid \Lambda \ell.x \mid \text{close}\{\sigma\} \mid \text{wait}\{\sigma\}$

$\mid \text{fork}\{\sigma_1\} \mid \text{fork}\{\sigma_1\}\{\sigma_2\}$

$\mid \text{send}\{\sigma_1\} \mid \text{send}\{\sigma_1\}[T] \mid \dots$

$\mid \text{receive}\{\sigma_1\} \mid \text{receive}\{\sigma_1\}[T] \mid \dots$

$\mid x\{\sigma\}$

Syntax: Expressions

$e ::= v \mid ee \mid (e, e) \mid \text{let } x = e \text{ in } e \mid \text{let } (x_1, x_2) = e \text{ in } e$ (Expressions)
 $\mid e; e \mid e[T] \mid e\{\sigma\} \mid \text{match } e \text{ with } \{\ell \rightarrow e_\ell\}_{\ell \in L}$
 $\mid \text{new } \mathcal{S} \mid \text{new } \mathcal{S}^\forall n_1 n_2$
 $\mid \text{inst}^\Psi x$

$\mathcal{C} ::= \phi\langle e \rangle \mid \mathcal{C} \parallel \mathcal{C} \mid (\nu xy^{\bar{\pi}}) \mathcal{C}$ (Configurations)

$\phi ::= \circ \mid \bullet$ (Thread flags)

$\Gamma ::= \epsilon \mid \Gamma, x : T$ (Typing contexts)

$\Psi ::= \epsilon \mid \Psi, x : T \mapsto \bar{\pi}$ (Priority maps)

Syntax: Expressions and Configurations

$e ::= v \mid ee \mid (e, e) \mid \text{let } x = e \text{ in } e \mid \text{let } (x_1, x_2) = e \text{ in } e$ (Expressions)
 $\mid e; e \mid e[T] \mid e\{\sigma\} \mid \text{match } e \text{ with } \{\ell \rightarrow e_\ell\}_{\ell \in L}$
 $\mid \text{new } \mathcal{S} \mid \text{new } \mathcal{S}^\forall n_1 n_2$
 $\mid \text{inst}^\Psi x$

$\mathcal{C} ::= \phi\langle e \rangle \mid \mathcal{C} \parallel \mathcal{C} \mid (\nu xy^{\bar{\pi}}) \mathcal{C}$ (Configurations)

$\phi ::= \circ \mid \bullet$ (Thread flags)

$\Gamma ::= \epsilon \mid \Gamma, x : T$ (Typing contexts)

$\Psi ::= \epsilon \mid \Psi, x : T \mapsto \bar{\pi}$ (Priority maps)

Main Results

Definition

$\mathcal{C}$ is **deadlock-free** if $\mathcal{C} \rightarrow^* \mathcal{C}' \not\rightarrow$ then $\mathcal{C}' \equiv \bullet\langle v \rangle$, where v is an unrestricted value.

Definition

$\mathcal{C}$ is in **canonical form** if it has one of the following two forms:

- ▶ $(\nu x_1 y_1^{\bar{\pi}_1}) \dots (\nu x_n y_n^{\bar{\pi}_n}) \bullet\langle e \rangle$
- ▶ $(\nu x_1 y_1^{\bar{\pi}_1}) \dots (\nu x_n y_n^{\bar{\pi}_n}) (\circ\langle e_1 \rangle \parallel \dots \parallel \circ\langle e_m \rangle \parallel \bullet\langle e \rangle)$, where no e_i is a value.

Main Results

Definition

$\mathcal{C}$ is **deadlock-free** if $\mathcal{C} \rightarrow^* \mathcal{C}' \not\rightarrow$ then $\mathcal{C}' \equiv \bullet\langle v \rangle$, where v is an unrestricted value.

Definition

$\mathcal{C}$ is in **canonical form** if it has one of the following two forms:

- ▶ $(\nu x_1 y_1^{\bar{\pi}_1}) \dots (\nu x_n y_n^{\bar{\pi}_n}) \bullet\langle e \rangle$
- ▶ $(\nu x_1 y_1^{\bar{\pi}_1}) \dots (\nu x_n y_n^{\bar{\pi}_n}) (\circ\langle e_1 \rangle \parallel \dots \parallel \circ\langle e_m \rangle \parallel \bullet\langle e \rangle)$, where no e_i is a value.

Theorem

Suppose $\mathcal{C}$ is a well-typed, closed configuration.

1. *If $\mathcal{C}$ is in canonical form, then $\mathcal{C} \rightarrow \mathcal{D}$ for some $\mathcal{D}$ or $\mathcal{C} \equiv \bullet\langle v \rangle$, where v is an unrestricted value.*
2. *$\mathcal{C}$ is deadlock-free.*

Conclusion

Concluding Remarks

- ▶ We have extended CFSTs for higher-order concurrent programs with mechanisms to enforce deadlock freedom.

Concluding Remarks

- ▶ We have extended CFSTs for higher-order concurrent programs with mechanisms to enforce deadlock freedom.
- ▶ The substantial expressiveness of CFSTs (polymorphic, non-regular recursion) does not conflict with strong correctness guarantees.
- ▶ Our extension still supports programs not expressible in regular session types. It is well-aligned with the FreeST programming language.

Concluding Remarks

- ▶ We have extended CFSTs for higher-order concurrent programs with mechanisms to enforce deadlock freedom.
- ▶ The substantial expressiveness of CFSTs (polymorphic, non-regular recursion) does not conflict with strong correctness guarantees.
- ▶ Our extension still supports programs not expressible in regular session types. It is well-aligned with the FreeST programming language.
- ▶ See arxiv for full definitions, proofs, and extra examples (Milner's scheduler).

Concluding Remarks

- ▶ We have extended CFSTs for higher-order concurrent programs with mechanisms to enforce deadlock freedom.
- ▶ The substantial expressiveness of CFSTs (polymorphic, non-regular recursion) does not conflict with strong correctness guarantees.
- ▶ Our extension still supports programs not expressible in regular session types. It is well-aligned with the FreeST programming language.
- ▶ See arxiv for full definitions, proofs, and extra examples (Milner's scheduler).

Future work includes inference mechanisms for defining priority sequences.

Deadlock-free Context-free Session Types

Jorge A. Pérez

University of Groningen (The Netherlands)

www.jperez.nl

joint work with

Andreia Mordido

University of Lisbon (Portugal)

(Based on a paper presented at FORTE 2026)

Appendix

Syntax of Types

$T, U ::=$	(Functional types)	$S ::=$	(Session types)
Unit	unit type	Skip	type skip
$T \rightarrow_{m}^{\rho_1, \rho_2} U$	function type	$!^{\rho} T$	send value of type T
$T \times U$	product type	$?^{\rho} T$	receive value of type T
α	(functional) type variable	$\oplus^{\rho} \{\ell : S_{\ell}\}_{\ell \in L}$	internal choice
$\mu \alpha. T$	rec. functional type	$\&^{\rho} \{\ell : S_{\ell}\}_{\ell \in L}$	external choice
$\forall \alpha^{\rho}. T$	poly. functional type	$S_1 ; S_2$	sequential composition
$\forall \beta^{\rho}. S$	poly. session type	Close^{ρ}	ready to be closed
$\mathbb{V}_{\iota}^F. T$	priority abstraction	Wait^{ρ}	wait to be closed
S	session type	β	(session) type variable
		$\mu \beta. S$	recursive session type
		$\mathbb{V}_{\iota}^S. S$	priority abstraction

Syntax of Types

$T, U ::=$	(Functional types)	$S ::=$	(Session types)
Unit	unit type	Skip	type skip
$T \rightarrow_m^{\rho_1, \rho_2} U$	function type	$!^\rho T$	send value of type T
$T \times U$	product type	$?^\rho T$	receive value of type T
α	(functional) type variable	$\oplus^\rho \{\ell : S_\ell\}_{\ell \in L}$	internal choice
$\mu\alpha. T$	rec. functional type	$\&^\rho \{\ell : S_\ell\}_{\ell \in L}$	external choice
$\forall\alpha^\rho. T$	poly. functional type	$S_1; S_2$	sequential composition
$\forall\beta^\rho. S$	poly. session type	Close^ρ	ready to be closed
$\forall^F_{\ell^\varphi}. T$	priority abstraction	Wait^ρ	wait to be closed
S	session type	β	(session) type variable
		$\mu\beta. S$	recursive session type
		$\forall^S_{\ell^\varphi}. S$	priority abstraction

Syntax of Functional and Session Types

$T, U ::=$	(Functional types)	$S ::=$	(Session types)
Unit	unit type	Skip	type skip
$T \rightarrow_m^{\rho_1, \rho_2} U$	function type	$!^\rho T$	send value of type T
$T \times U$	product type	$?^\rho T$	receive value of type T
α	(functional) type variable	$\oplus^\rho \{\ell: S_\ell\}_{\ell \in L}$	internal choice
$\mu\alpha. T$	rec. functional type	$\&^\rho \{\ell: S_\ell\}_{\ell \in L}$	external choice
$\forall\alpha^\rho. T$	poly. functional type	$S_1; S_2$	sequential composition
$\forall\beta^\rho. S$	poly. session type	Close^ρ	ready to be closed
$\forall^F_{\ell^\varphi}. T$	priority abstraction	Wait^ρ	wait to be closed
S	session type	β	(session) type variable
		$\mu\beta. S$	recursive session type
		$\forall^S_{\ell^\varphi}. S$	priority abstraction

Types for Constants

send : $\forall \ell_1^{(\perp, \top]}. \forall \alpha^{\ell_1}. \forall \ell_2^{(\perp, \ell_1)}. \forall \ell_3^{(\ell_2, \top]}. \forall \beta^{\ell_3}. \alpha \rightarrow_{\text{un}}^{\top, \perp} !^{\ell_2} \alpha; \beta \rightarrow_{\text{lin}}^{\top, \ell_2} \beta$

receive : $\forall \ell_1^{(\perp, \top]}. \forall \alpha^{\ell_1}. \forall \ell_2^{(\perp, \ell_1)}. \forall \ell_3^{(\ell_2, \top]}. \forall \beta^{\ell_3}. ?^{\ell_2} \alpha; \beta \rightarrow_{\text{un}}^{\top, \ell_2} \alpha \times \beta$

fork : $\forall \ell_1^{(\perp, \top]}. \forall \ell_2^{[\perp, \top]}. (\text{Unit} \rightarrow_{\text{un}}^{\ell_1, \ell_2} \text{Unit}) \rightarrow_{\text{un}}^{\top, \perp} \text{Unit}$

fix : $\forall \ell^{[\perp, \top]}. \forall \alpha^{\ell}. ((\alpha \rightarrow_{\text{un}}^{\top, \perp} \alpha) \rightarrow_{\text{un}}^{\top, \perp} (\alpha \rightarrow_{\text{un}}^{\top, \perp} \alpha)) \rightarrow_{\text{un}}^{\top, \perp} (\alpha \rightarrow_{\text{un}}^{\top, \perp} \alpha)$

close : $\forall \ell^{(\perp, \top]}. \text{Close}^{\ell} \rightarrow_{\text{un}}^{\top, \ell} \text{Unit}$

wait : $\forall \ell^{(\perp, \top]}. \text{Wait}^{\ell} \rightarrow_{\text{un}}^{\top, \ell} \text{Unit}$

() : Unit

Types for Constants with Priority Abstractions

send : $\forall \ell_1^{(\perp, \top]}. \forall \alpha^{\ell_1}. \forall \ell_2^{(\perp, \ell_1)}. \forall \ell_3^{(\ell_2, \top]}. \forall \beta^{\ell_3}. \alpha \rightarrow_{\text{un}}^{\top, \perp} !^{\ell_2} \alpha; \beta \rightarrow_{\text{lin}}^{\top, \ell_2} \beta$

receive : $\forall \ell_1^{(\perp, \top]}. \forall \alpha^{\ell_1}. \forall \ell_2^{(\perp, \ell_1)}. \forall \ell_3^{(\ell_2, \top]}. \forall \beta^{\ell_3}. ?^{\ell_2} \alpha; \beta \rightarrow_{\text{un}}^{\top, \ell_2} \alpha \times \beta$

fork : $\forall \ell_1^{(\perp, \top]}. \forall \ell_2^{[\perp, \top]}. (\text{Unit} \rightarrow_{\text{un}}^{\ell_1, \ell_2} \text{Unit}) \rightarrow_{\text{un}}^{\top, \perp} \text{Unit}$

fix : $\forall \ell^{[\perp, \top]}. \forall \alpha^{\ell}. ((\alpha \rightarrow_{\text{un}}^{\top, \perp} \alpha) \rightarrow_{\text{un}}^{\top, \perp} (\alpha \rightarrow_{\text{un}}^{\top, \perp} \alpha)) \rightarrow_{\text{un}}^{\top, \perp} (\alpha \rightarrow_{\text{un}}^{\top, \perp} \alpha)$

close : $\forall \ell^{(\perp, \top]}. \text{Close}^{\ell} \rightarrow_{\text{un}}^{\top, \ell} \text{Unit}$

wait : $\forall \ell^{(\perp, \top]}. \text{Wait}^{\ell} \rightarrow_{\text{un}}^{\top, \ell} \text{Unit}$

() : Unit

Configuration Reduction (Sample)

Let $\mathcal{F}, \mathcal{F}'$ denote evaluation contexts.

- ▶ Endpoint creation, without polymorphism:

$$\mathcal{F}[\text{new } \mathcal{S}] \rightarrow (\nu xy^{\epsilon}) \mathcal{F}[(x, y)]$$

where $x, y \notin \text{fv}(\mathcal{F})$ and $\text{unr}(\mathcal{S}) \neq \text{Skip}$.

Configuration Reduction (Sample)

Let $\mathcal{F}, \mathcal{F}'$ denote evaluation contexts.

- ▶ Endpoint creation, without polymorphism:

$$\mathcal{F}[\text{new } \mathcal{S}] \rightarrow (\nu xy^{\epsilon}) \mathcal{F}[(x, y)]$$

where $x, y \notin \text{fv}(\mathcal{F})$ and $\text{unr}(\mathcal{S}) \neq \text{Skip}$.

- ▶ Endpoint creation, with $\mathcal{S}$ polymorphic:

$$\mathcal{F}[\text{new } \mathcal{S}^{\forall} \mathbf{n}_1 \mathbf{n}_2] \rightarrow (\nu xy^{\bar{\pi}}) \mathcal{F}[(x, y)]$$

where $x, y \notin \text{fv}(\mathcal{F})$ and $\bar{\pi} = [\mathbf{n}_1 + k \cdot \mathbf{n}_2 \mid k \in \mathbb{N}]$.

Configuration Reduction (Sample)

Let $\mathcal{F}, \mathcal{F}'$ denote evaluation contexts.

- ▶ Endpoint creation, without polymorphism:

$$\mathcal{F}[\text{new } \mathcal{S}] \rightarrow (\nu xy^\epsilon) \mathcal{F}[(x, y)]$$

where $x, y \notin \text{fv}(\mathcal{F})$ and $\text{unr}(\mathcal{S}) \neq \text{Skip}$.

- ▶ Endpoint creation, with $\mathcal{S}$ polymorphic:

$$\mathcal{F}[\text{new } \mathcal{S}^\forall \mathbf{n}_1 \mathbf{n}_2] \rightarrow (\nu xy^{\bar{\pi}}) \mathcal{F}[(x, y)]$$

where $x, y \notin \text{fv}(\mathcal{F})$ and $\bar{\pi} = [\mathbf{n}_1 + k \cdot \mathbf{n}_2 \mid k \in \mathbb{N}]$.

- ▶ Synchronization:

$$\begin{aligned} & (\nu xy^{\bar{\pi}}) (\mathcal{F}[\text{send}\{_\}[_\]\{_\}\{_\}[_\]vx] \parallel \mathcal{F}'[\text{receive}\{_\}[_\]\{_\}\{_\}[_\]y]) \\ & \rightarrow (\nu xy^{\bar{\pi}}) (\mathcal{F}[x] \parallel \mathcal{F}'[(v, y)]) \end{aligned}$$

Typing Judgments

- For **types**:

$$\Delta \mid \Theta \vdash T$$

T is well-formed under the polymorphic context Δ and the priority context Θ .

Typing Judgments

- ▶ For **types**:

$$\Delta \mid \Theta \vdash T$$

T is well-formed under the polymorphic context Δ and the priority context Θ .

- ▶ For **expressions**:

$$\Delta \mid \Gamma \mid \Theta \mid \Psi \vdash e : T \diamond \bar{\pi} \diamond \rho$$

e has type T and latent effect ρ under the typing environments Δ, Γ, Θ , and Ψ .
The evaluation of e might yield a priority sequence $\bar{\pi}$.

- ▶ For **configurations**:

$$\Gamma \vdash_{\Psi}^{\phi} \mathcal{C}$$

$\mathcal{C}$ is well-typed under Γ with priority map Ψ .

Main Results

Lemma (Type Preservation: Expressions)

If $\Delta \mid \Gamma \mid \Theta \mid \Psi \vdash e_1 : T \diamond \bar{\pi} \diamond \rho$ and $e_1 \rightarrow e_2$ then $\Delta \mid \Gamma \mid \Theta \mid \Psi \vdash e_2 : T \diamond \bar{\pi} \diamond \rho$.

Theorem (Type Preservation: Configurations)

If $\Gamma \vdash_{\Psi}^{\phi} C_1$ and $C_1 \rightarrow C_2$ then $\Gamma \vdash_{\Psi}^{\phi} C_2$.

Theorem (Progress for Terms)

If $\Delta \mid \Gamma \mid \Theta \mid \Psi \vdash e : T \diamond \bar{\pi} \diamond \rho$ and for all $x : T \in \Gamma$, T is a session type, then:

- ▶ e is a value, or
- ▶ $e \rightarrow e'$ for some e' , or
- ▶ e is ready.